Using the get_row() method

```
$userdata = $wpdb->get_row("SELECT * FROM ' . $wpdb->users
. ' WHERE ID =43");
$user_assocarray = $wpdb->get_row("SELECT * FROM ' .
$wpdb->users . ' WHERE ID = 43", ARRAY_A);
$user_numarray = $wpdb->get_row("SELECT * FROM ' . $wpdb-
>users . ' WHERE ID= 43", ARRAY_N);
```

Retrieving a full data set

If you have to do raw SQL queries, try to limit the scope of what data you are actually retrieving from the database. If you can just retrieve a column of data, then use $wpdb->get_col(). If you only need a single value, consider the $wpdb->get_var() method.

However, sometimes you will need an entire dataset. Fortunately, you have the $wpdb->get_results() method. Like the $wpdb->get_col() and $wpdb->get_row() methods, you can pass ARRAY_A or ARRAY_N as a flag so the returned data comes back as an array.

Using the get_results() method

```
$users = $wpdb->get_results("SELECT * FROM ' . $wpdb-
>users . ' WHERE user_registered > '2009-07-01'");
$users_assocarr = $wpdb->get_results("SELECT * FROM ' .
$wpdb->users . 'WHERE user_registered > '2009-07-01'",
ARRAY_A);
$users_numarr = $wpdb->get_results("SELECT * FROM ' .
$wpdb->users . ' WHERE user_registered > '2009-07-01'",
ARRAY_N);
```

Performing other queries

The WordPress database class provides a $wpdb->query() method for other types of SQL queries. For example, you can use it to get a listing of all tables in the database (something that might be helpful because other plugins might supply their own tables):

```
$wpdb->query("INSERT INTO {$wpdb->prefix}my_plugin_table SET field1
=
'$safe_value', field2 = '$safe_value2' WHERE field3 = 'some
value'");
```

You could also use this method for INSERT, DELETE, or UPDATE SQL queries if your plugin provides its own tables.